Capa de transporte: comparación entre TCP y UDP

Material educativo de ingeniería de redes. Contenido original de libre uso educativo.

1. La capa de transporte

La capa de transporte, cuarta del modelo OSI, proporciona comunicación lógica entre procesos que se ejecutan en equipos distintos. La capa de red entrega paquetes de un host a otro mediante direcciones IP, pero no sabe a qué aplicación pertenece cada paquete. La capa de transporte resuelve ese problema con los números de puerto: la combinación de dirección IP y puerto, llamada socket, identifica de forma única a cada extremo de una comunicación. Los dos protocolos principales son TCP (Transmission Control Protocol), orientado a conexión y fiable, y UDP (User Datagram Protocol), sin conexión y ligero. La elección entre ambos es una decisión de diseño que equilibra fiabilidad y latencia.

2. TCP: transporte fiable orientado a conexión

2.1 Saludo de tres vías

TCP establece una conexión antes de intercambiar datos mediante el saludo de tres vías (three-way handshake). En el primer paso, el cliente envía un segmento con el indicador SYN activado y un número de secuencia inicial aleatorio. En el segundo, el servidor responde con un segmento SYN-ACK: con el indicador ACK confirma el número del cliente (sumando uno) y con SYN anuncia su propio número de secuencia inicial. En el tercero, el cliente envía un ACK que confirma el número del servidor. Tras estos tres segmentos, ambos extremos tienen sincronizados sus contadores y la conexión pasa al estado ESTABLISHED. El cierre ordenado utiliza segmentos FIN y ACK en cada sentido, y un segmento RST permite abortar la conexión de inmediato.

2.2 Acuses de recibo y retransmisión por timeout

TCP numera cada byte del flujo de datos. El receptor devuelve acuses de recibo (ACK) que indican el siguiente byte que espera recibir. Son acuses acumulativos: un ACK con valor 1001 significa que todos los bytes hasta el 1000 llegaron correctamente. Cuando el emisor envía un segmento, inicia un temporizador de retransmisión. Si expira el timeout (RTO) sin que llegue el ACK correspondiente, el emisor asume que el segmento se perdió y lo retransmite. El RTO no es fijo: se calcula a partir del tiempo de ida y vuelta (RTT) medido, mediante un promedio suavizado y su desviación. Además, la recepción de tres ACK duplicados activa la retransmisión rápida, que reenvía el segmento perdido sin esperar a que venza el temporizador.

2.3 Ventana deslizante y control de flujo

Para no saturar al receptor, TCP aplica control de flujo mediante una ventana deslizante. En cada segmento, el receptor anuncia su ventana de recepción (rwnd): la cantidad de bytes que aún puede almacenar en su búfer. El emisor puede mantener en tránsito, sin confirmar, como máximo esa cantidad de datos. Cada vez que llega un ACK, la ventana se desliza hacia adelante y habilita el envío de nuevos bytes, lo que permite transmitir varios segmentos de forma continua sin esperar una confirmación por cada uno. Si el receptor anuncia una ventana de cero, el emisor detiene el envío y sondea periódicamente hasta que se libere espacio.

2.4 Control de congestión

El control de congestión protege a la red, no al receptor. El emisor mantiene una ventana de congestión (cwnd) y envía, como máximo, el menor valor entre cwnd y rwnd. Al iniciar, el algoritmo de arranque lento (slow start) parte de un valor pequeño y duplica cwnd en cada RTT hasta alcanzar un umbral llamado ssthresh. Después entra en evitación de congestión, donde cwnd crece de forma lineal, aproximadamente un segmento por RTT. Si se detecta una pérdida, TCP interpreta que la red está congestionada y reduce su ritmo: tras un timeout, cwnd vuelve a un valor mínimo y ssthresh baja a la mitad; tras tres ACK duplicados, cwnd se reduce a la mitad. Variantes modernas como CUBIC y BBR refinan este comportamiento.

3. UDP: transporte ligero sin conexión

3.1 Cabecera de 8 bytes

UDP, definido en la RFC 768, es deliberadamente mínimo. Su cabecera contiene solo cuatro campos de 16 bits: puerto de origen, puerto de destino, longitud y suma de verificación (checksum), para un total de 8 bytes. La cabecera TCP, en cambio, ocupa como mínimo 20 bytes y puede llegar a 60 con opciones, pues incluye números de secuencia y de acuse, longitud de cabecera, indicadores (SYN, ACK, FIN, RST, PSH, URG), ventana, checksum y puntero urgente. Esa diferencia importa cuando los paquetes son pequeños y numerosos, como las muestras de audio de una llamada. Además, UDP no mantiene estado de conexión, lo que reduce el consumo de memoria y procesamiento.

3.2 Entrega best-effort

UDP ofrece una entrega best-effort ("mejor esfuerzo"): envía cada datagrama sin garantías. No hay saludo inicial, acuses de recibo, retransmisión, ordenamiento ni control de congestión. Cada datagrama es independiente, por lo que puede perderse, duplicarse o llegar en distinto orden. El checksum permite detectar corrupción, pero el datagrama dañado simplemente se descarta, sin aviso al emisor. Si una aplicación necesita fiabilidad sobre UDP, debe implementarla por su cuenta.

4. Comparación y criterios de elección

Característica	TCP	UDP
Conexión	Sí, saludo de tres vías	No
Fiabilidad	ACK y retransmisión	Best-effort
Orden de entrega	Garantizado	No garantizado
Cabecera	20 a 60 bytes	8 bytes
Flujo y congestión	Sí	No
Latencia	Mayor	Menor
Usos típicos	Web, archivos, correo	Voz, video, juegos, DNS

4.1 Por qué UDP en tiempo real

En videollamadas, VoIP, streaming en vivo y videojuegos multijugador, los datos tienen una vida útil muy corta. Un paquete de audio que llega con 400 ms de retraso ya no sirve, así que retransmitirlo es inútil. TCP, al garantizar entrega ordenada, retiene los datos posteriores hasta recuperar el segmento perdido (bloqueo de cabeza de línea), y el usuario percibe congelamientos y saltos. Con UDP, la aplicación descarta lo perdido y continúa con lo más reciente. En un juego en línea, la posición actual de un jugador reemplaza a la anterior, por lo que reenviar el estado viejo carece de valor. A esto se suma la ausencia de handshake y de control de congestión, que reducen la latencia y el jitter.

4.2 Por qué TCP en archivos y web

En la transferencia de archivos y la navegación web ocurre lo contrario: cada byte importa. Un instalador al que le falta un fragmento o una página con bloques desordenados queda inutilizable, y el usuario prefiere esperar unos milisegundos más antes que recibir datos erróneos. Por eso HTTP y HTTPS (versiones 1.1 y 2), FTP, SFTP y los protocolos de correo SMTP e IMAP se apoyan en TCP. No es una regla absoluta: HTTP/3 funciona sobre QUIC, que se construye sobre UDP e incorpora fiabilidad y cifrado propios, y DNS usa UDP para consultas breves.

5. Conclusión

TCP y UDP no compiten: resuelven necesidades distintas. TCP aporta conexión, orden, fiabilidad, ventana deslizante y control de congestión, a cambio de mayor cabecera y latencia. UDP ofrece una cabecera de 8 bytes y entrega best-effort, trasladando a la aplicación las decisiones de fiabilidad. Al diseñar o diagnosticar un servicio, la pregunta clave es si conviene más recibir todo correctamente, aunque tarde, o recibir a tiempo, aunque se pierda algo.